TRACE Project Handover: What We Are Doing and How to Continue

Oct 12, 2026 · @Optimus

This document explains the TRACE research project to a new team member: what we are building, where everything is, and exactly what to do next, with the input, output and finish line of every step. Section 3 lists the upgrades adopted after the October 2026 review of new research and models.

1. What the Project Does

TRACE turns photos of handwritten exam answers from the Structured Programming Language (C) course into text, then grades that text against the official marking scheme. It must run on one ordinary PC, and every mark it suggests must point to the exact words the student wrote.

The problem we found

Modern AI models that read handwriting (vision-language OCR models) are trained to produce fluent, correct-looking text. When a student forgets a semicolon or writes `i <= n` instead of `i < n`, the model may quietly write the *correct* version instead. We call this **silent correction**.

Silent correction is dangerous for grading: the grader then gives marks for a mistake the student actually made. It is also hard to notice, because the usual accuracy score (character error rate) can look excellent while it happens.

The two stages

1. **Stage 1, faithful transcription.** A model reads each page photo and writes exactly what is on it, mistakes included, as a Markdown text file. We test several models and pick the one that keeps student errors best.
2. **Stage 2, grounded grading.** A second model reads that text, looks up the relevant parts of our C reference booklet and the marking scheme, and suggests marks. For every mark it must quote the student's own words; a checker rejects any quote that is not really in the text. Unclear cases go to a human examiner.

The two stages never share a model. The grader only sees text, never the photo, so every decision can be traced back.

The hardware rule

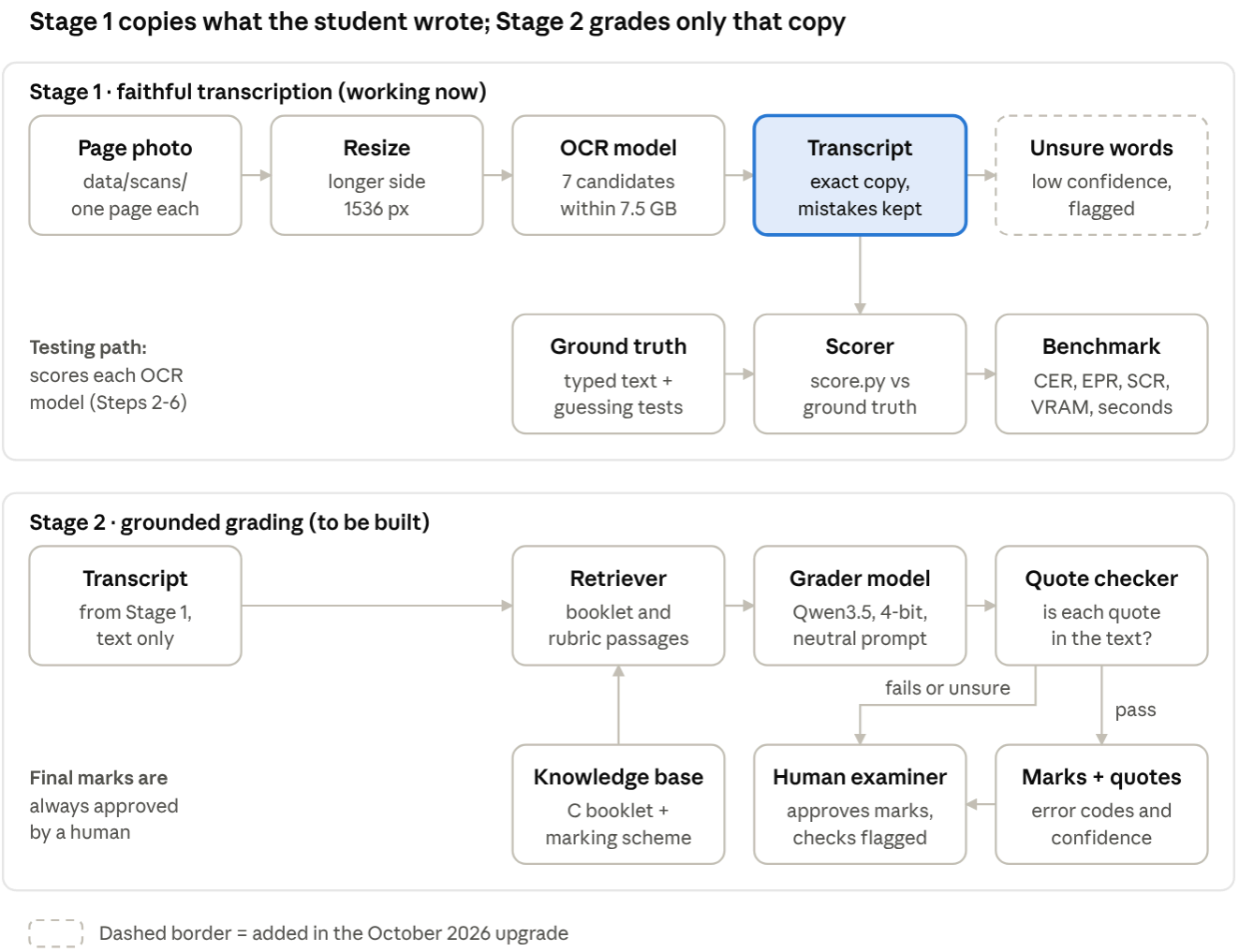
Our PC has an RTX 5070 with 12 GB of GPU memory. The finished system must work on an **8 GB** GPU, so every test is run with an 8 GB budget (in practice 7.5 GB, for safety). We check this with a memory-logging script on every run. Fine-tuning may use the full 12 GB.

The four research questions

#	Question	How we answer it
RQ1	How often do OCR models silently correct student errors, and does the normal accuracy score hide it?	Compare clean pages with copies containing planted mistakes (Stage 1)
RQ2	Does giving the grader our course booklet improve agreement with human examiners?	Grade real answers with and without the booklet (Stage 2)
RQ3	Does forcing the grader to quote evidence reduce unsupported marks?	Count how many quotes are genuine, before and after the checker
RQ4	Can the whole system run within 8 GB at a practical speed?	Memory and time logs from every run

The final output is a working demo and a conference paper. Most of the paper's numbers will come from the steps in this document.

2. Pipeline at a Glance



TRACE pipeline · Stage 1 working, Stage 2 to be built

The highlighted transcript is the hand-off between the stages: Stage 1 must copy the page faithfully, mistakes included, because Stage 2 never sees the photo. The lower row of Stage 1 is only for testing: it compares each model's transcript with the typed ground truth to choose the best transcriber. Dashed boxes are the October 2026 additions explained in Section 3.

3. What Is New: October 2026 Upgrades

We reviewed the research and model releases from January to September 2026 and adopted nine upgrades: newer models, four new measurements and a stronger grader design. In the pipeline diagram, dashed boxes mark where they enter.

Why the project is still new

Two earlier studies already saw OCR models "correct" what they read: one in historical documents (described, not measured) and one in 11 handwritten Python programs read by GPT-4. Our contribution is narrower and still open: the first controlled, error-type-by-error-type measurement of silent correction in handwritten C answers, with models that run locally, and its effect on marks.

The risk is real. A 2026 classroom study at the University of Illinois found that transcription errors caused 87% of an AI grader's disagreements with human graders, and its errors favoured students about 2.5 to 1.

The nine upgrades

#	Upgrade	What changes	Why (2026 evidence)	Used in
1	Newer transcribers	Add PaddleOCR-VL 1.5, Qwen3.5-4B, Gemma 4 E4B and Chandra OCR 2; drop Qwen2.5-VL-3B	2026 releases lead the document-OCR benchmarks; Qwen2.5-VL is two generations old	Step 4
2	Newer grader	Qwen3.5-9B or Qwen3.5-4B in 4-bit instead of Qwen3-8B; Gemma 4 E4B as a second option	Released March and April 2026 under Apache 2.0; both also read images, which upgrade 6 needs	Step 11
3	Guessing-test pages	3 or 4 extra sheets of deliberately strange code: made-up names (`qzvx`), misspelled keywords (`whlie`)	2026 studies show OCR vision models replace unfamiliar text with likely words; code was never tested	Step 10a
4	Confidence flags	The transcriber also saves how sure it was of each word; unsure words are highlighted for the examiner	The 2026 OCR-hallucination studies recommend this to catch errors that accuracy scores hide	Steps 2 and 10b
5	Neutral grading prompt	No harsh instructions such as "never give partial credit"	A September 2026 study: one such line pushed 14 of 17 open models out of acceptable accuracy	Step 11

#	Upgrade	What changes	Why (2026 evidence)	Used in
6	One-call baseline	A vision model grades straight from the photo, compared with our two-stage pipeline	The Illinois system grades in one call; reviewers will ask why we split the stages	Step 12
7	Fine-tuned grader (optional)	A small LoRA add-on trained on graded answers: a public CS-exam dataset plus ours	The same September study: one add-on trained on about 3,900 graded answers brought five small open models to human-level agreement	Step 11
8	Safe-automation curve	The grader reports its confidence; we measure accuracy against the share of answers graded without a human	CHiL(L)Grader (March 2026) graded 35–65% of answers automatically at expert-level agreement; departments need this number before adopting	Step 12
9	Fairness check	Report under-marking (hurts the student) separately from over-marking, and compare neat with untidy handwriting	A June 2026 fairness study bases its case for automatic grading on exactly this split	Steps 2 and 12

Upgrades 1, 3 and 9 need no new code from you: run the new models like the others and write the extra sheets like Step 1. The project lead will provide the updated runner scripts for upgrade 4.

Real-world fit

Everything still runs on one PC within 8 GB and keeps student data on campus. That suits Bangladesh's draft National AI Policy 2026–2030, which requires impact assessments for high-risk uses by public bodies, and the Personal Data Protection Ordinance 2025. PaddleOCR-VL also lists Bengali support, which opens a later extension to Bangla–English answers.

4. Where Things Are

Everything lives under `~/research` on the lab PC (Ubuntu 24.04, RTX 5070 12 GB, 64 GB RAM). Never store project files anywhere else.

```
~/research/
├── code/                    git repository: all scripts
│   ├── vram_guard.py        GPU memory logger (PASS/FAIL against 7.5 GB)
│   ├── score.py             scores transcripts against the ground truth
│   ├── ocr/run_glm.py       runs GLM-OCR on one image or a folder
│   └── testset_source/      pages.py (test-page text) and the generators
├── data/
│   ├── scans/               20 clean test-page photos (p01.jpg ...)
│   ├── planted/             photos of the 8 mistake copies + manifest.json
│   ├── ground_truth/        typed text of every test page (p01.md ...)
│   ├── pages.csv            list of all 28 test pages
│   └── real/                the 15 real exam scripts (created in Step 7)
├── docs/writing_sheets.pdf  printable sheets for writing the test pages
├── models/hf, models/gguf   downloaded models (grader: Qwen3-8B for now)
├── envs/                    one Python environment per model family
├── results/                 one folder per experiment run (never overwrite)
├── logs/                    VRAM traces from vram_guard.py
├── llama.cpp/               built llama.cpp (llama-server for the grader)
└── paper/                   keep the paper draft here (trace_paper.tex)
```

Python environments

Each model family needs different library versions, so each has its own environment. Activate the right one before running anything: `source ~/research/envs/<name>/bin/activate`.

Environment	Used for
`ocr-glm`	GLM-OCR and the new 2026 models (PaddleOCR-VL 1.5, Qwen3.5-4B, Gemma 4 E4B) if their library versions agree; runs (`run_glm.py`), `score.py` , `vram_guard.py`
`ocr-deepseek`	DeepSeek-OCR-2
`ocr-paddle`	PaddleOCR (the non-generative comparison model)
`grader`	Stage 2: retrieval, grading client, evaluation
`train`	Fine-tuning (later)
`vllm`	Serving GLM-OCR for its official SDK (Step 3)

Scripts: what goes in and what comes out

Script	Input	Output
`vram_guard.py`	A command to run (after `--`), or `--pid` of a running server	A CSV memory trace in `logs/` and a final `PASS` or `FAIL` line
`ocr/run_glm.py`	`--images` (one photo or a folder), `--out` folder, `--max-side 1536`	One `<page>.md` transcript per photo, `runs.csv` (time, tokens, memory), resized copies
`score.py`	`--data ../data` and one or more `--pred` results folders	`scores_summary.json`, `scores_pages.csv`, `scores_errors.csv` in each results folder
`testset_source/build_gt.py`	`pages.py`	Rebuilds `data/ground_truth/`, `manifest.json` and `pages.csv`

File naming

- Test pages: `p01` to `p20`; mistake copies `p01_err`, `p03_err`, `p07_err`, `p08_err`, `p09_err`, `p11_err`, `p13_err`, `p16_err`.
- Real scripts: `s01_p01.jpg` means script 01, page 01.
- Results folders: `<model>_<resolution>`, for example `glm_1536`, `paddlevl_1536`, `glm_sweep_2048`.
- A transcript always has the same name as its photo: `p07.jpg` gives `p07.md`.

Related documents

- Research PC Setup: Ubuntu and RTX 5070 for the TRACE Experiments: how the PC was set up, the 8 GB memory rules, and the full `vram_guard.py` code.
- Structured Programming in C: A Complete Reference: the C reference booklet that Stage 2 will use as its knowledge base, including the student error codes in Appendix A.
- Paper draft (`trace_paper.tex` and `.pdf`) from the project lead: red text marks the numbers your results will fill in.

5. Work Plan

The first five rows are finished. Your work starts at Step 1 and mostly runs in order: each step's **Needs** column says what must be finished first. Update the Status column as you go.

Step	Task	Owner	Status	Needs
-	PC, drivers and Python environments set up	Project lead	Done	-
-	GLM-OCR tested on two real pages (test01, test02)	Project lead	Done	-
-	Test-page sheets, typed ground truth and `score.py` prepared	Project lead	Done	-
-	C reference booklet drafted	Project lead	Done	-
-	Paper draft with placeholder numbers	Project lead	Done	-
1	Write and photograph the 28 test pages	Assistant	In progress	`writing_sheets.pdf`
2	Transcribe all test pages with GLM-OCR and score them	Assistant	Not started	Step 1
3	Run GLM-OCR through its official SDK (layout detection)	Assistant	Not started	Step 1
4	Benchmark the 2026 OCR models the same way (list in Step 4)	Assistant (runner scripts from project lead)	Not started	Step 1

Step	Task	Owner	Status	Needs
5	Resolution sweep: memory and faithfulness at 5 image sizes	Assistant	Not started	Steps 2 and 4
6	Choose the transcriber with the decision rule	Project lead with assistant	Not started	Steps 2 to 5
7	Prepare the 15 real exam scripts	Assistant	Not started	Permission from the teacher
8	Independent second grading of the real scripts	Instructor (arranged by project lead)	Not started	Step 7
9	Instructor review of the C booklet	Instructor	Not started	-
10	Extra tests: guessing-test pages, confidence flags, fairness check	Project lead (scripts), assistant (writing)	Not started	Step 1
11	Build the Stage 2 grader (Qwen3.5, neutral prompt, optional LoRA)	Project lead	Not started	Steps 6 and 9
12	Grade the real answers and compare with the instructors	Project lead with assistant	Not started	Steps 8 and 11
13	Put the final numbers into the paper	Project lead	Not started	Steps 6 and 12

6. Stage 1 Steps: Testing the Transcribers

The goal of Steps 1 to 6 is one decision: which OCR model copies student handwriting most faithfully within 8 GB. Every model gets exactly the same photos, image size and scoring, so the comparison is fair.

Start every work session with:

```
cd ~/research/code
source ~/research/envs/ocr-glm/bin/activate
git status                # see what changed since last time
```

Step 1: Write and photograph the 28 test pages

Input: the printed `docs/writing_sheets.pdf`, exam-style paper, one blue or black ballpoint pen.

Do:

1. Read the cover page of the PDF first. Copy each sheet onto one page exactly: same words, symbols, line breaks and indentation.
2. On the 8 sheets marked "deliberate mistakes", copy the red underlined text exactly as shown. Never copy the small red numbers or the red box.
3. Never fix anything. If you make an accidental mistake, start that page again on a fresh sheet.
4. Compare each page with its sheet word by word and tick the tracking sheet.
5. Photograph every page with the same phone, in daylight, straight from above, whole page in frame, no shadows or fingers.
6. Move the photos to the PC by USB cable or Google Drive. Do not use WhatsApp or Messenger, which compress images.

Output:

- `data/scans/p01.jpg` to `p20.jpg` (20 photos)
- `data/planted/p01_err.jpg`, `p03_err.jpg`, `p07_err.jpg`, `p08_err.jpg`, `p09_err.jpg`, `p11_err.jpg`, `p13_err.jpg`, `p16_err.jpg` (8 photos)
- The filled tracking sheet

Done when:

```
ls ~/research/data/scans/*.jpg | wc -l      # prints 20
ls ~/research/data/planted/*.jpg | wc -l    # prints 8
```

If a page differs from its sheet (a word changed, a line break moved), the ground truth must be changed to match the page, not the other way round:

```
cd ~/research/code/testset_source
nano pages.py          # edit that page's text
python build_gt.py     # rebuilds ground_truth/, manifest.json and pages.csv
```

Never edit the `*_err.md` files by hand: the mistake list stores exact character positions.

Step 2: Transcribe all test pages with GLM-OCR and score them

Input: the 28 photos from Step 1.

Do:

```
cd ~/research/code
source ~/research/envs/ocr-glm/bin/activate
python vram_guard.py --budget-gb 7.5 --log ../logs/glm_1536_scans.csv -- \
  python ocr/run_glm.py --images ../data/scans --out ../results/glm_1536
python vram_guard.py --budget-gb 7.5 --log ../logs/glm_1536_planted.csv -- \
  python ocr/run_glm.py --images ../data/planted --out ../results/glm_1536
python score.py --data ../data --pred ../results/glm_1536 --names GLM-OCR
```

Output (in `results/glm_1536/`):

- 28 transcripts (`p01.md` … `p16_err.md`), `runs.csv` with time and memory per page, and resized copies of the photos
- `scores_summary.json`, `scores_pages.csv`, `scores_errors.csv`
- Two memory logs in `logs/`, each ending with a `[vram_guard] ... PASS` or `FAIL` line

Done when:

- `score.py` prints `pages scored 28, missing 0`.
- You opened `scores_errors.csv` and checked by eye every row whose outcome is `corrupted` or whose `low_confidence` is 1. The automatic check is about 97% reliable, so these rows need a human look.
- You wrote a short `results/glm_1536/notes.md`: the two memory lines, the main scores, and anything strange you saw (dropped braces, flattened tables, invented words).

How to read the scores:

Score	Meaning	Better is
CER	Share of characters wrong on clean pages	Lower
CER (no spaces)	Same, ignoring spacing and indentation	Lower
WER	Share of words wrong	Lower
EPR	Share of planted mistakes copied exactly	Higher
SCR	Share of planted mistakes silently fixed	Lower
`{` `}` `;` counts	Symbols in the transcript vs the ground truth	Equal

Watch out: `runs.csv` adds rows every time, so never rerun into the same folder. Delete the folder first or use a new name. The first page of each run includes start-up time, so report the median seconds per page.

Step 3: Run GLM-OCR through its official SDK

The plain run in Step 2 reads tables column by column, which breaks "differentiate" answers. GLM-OCR's official SDK first finds text, table and formula regions, then reads each with the right prompt. We test whether that copies tables better.

Input: the resized photos from Step 2 (`results/glm_1536/resized_1536/`), so both runs see identical images.

Do:

1. Terminal 1, start the model server with a memory cap:

```
source ~/research/envs/vllm/bin/activate
vllm serve zai-org/GLM-OCR --port 8080 --served-model-name glm-ocr \
  --gpu-memory-utilization 0.55 --max-model-len 16384 --max-num-seqs 4 \
  --allowed-local-media-path /
```

2. Terminal 2, install the SDK once and copy its settings file:

```
uv venv ~/research/envs/glmocr-sdk --python 3.12
source ~/research/envs/glmocr-sdk/bin/activate
uv pip install "glmocr[selfhosted]"
git clone https://github.com/zai-org/glm-ocr.git ~/research/glm-ocr
cp ~/research/glm-ocr/glmocr/config.yaml ~/research/code/ocr/glmocr_sdk.yaml
```

3. In `glmocr_sdk.yaml` change only: `maas: enabled` to `false`; under `ocr_api`, `api_port` to `8080` and `model` to `glm-ocr`; `temperature` to `0.0`; layout detection on. If the setting names look different, ask the project lead.
4. Terminal 3, start the memory log, then run the SDK in Terminal 2:

```
# Terminal 3
source ~/research/envs/ocr-glm/bin/activate
python ~/research/code/vram_guard.py --pid $(pgrep -f "vllm serve") --
duration 900 \
  --log ~/research/logs/glm_sdk_vram.csv
# Terminal 2
cd ~/research/code
glmocr parse ../results/glm_1536/resized_1536/ --config ocr/glmocr_sdk.yaml \
  --layout-device cpu --output ../results/glm_sdk_raw/
```

5. Copy each page's Markdown output to `results/glm_sdk_1536/<page>.md` (for example `p02.md`), then score it like Step 2.

Output: `results/glm_sdk_1536/` with 28 transcripts and score files, plus `logs/glm_sdk_vram.csv`.

Done when: scored with 0 missing pages, and `notes.md` compares the table pages (p02, p06, p13 to p16) with Step 2.

Watch out: vLLM reserves its memory share up front, so the memory log shows that reservation (about 6.5 GB), not what the model needs. That is expected.

Step 4: Benchmark the other OCR models

Input: the same 28 photos at 1536 px, and one runner script per model from the project lead. Each runner takes the same options as `run_glm.py` and writes the same output files.

Model	Size	Why it is in the test	Results folder
GLM-OCR (Steps 2 and 3)	0.9B	Ranked first on OmniDocBench at its 2026 launch	`glm_1536`, `glm_sdk_1536`
PaddleOCR-VL 1.5 (new)	0.9B	Strongest small document model of 2026; supports Bengali	`paddlevl_1536`
DeepSeek-OCR-2	3B	A 2026 study shows it relies on language guessing	`deepseek_1536`
Qwen3.5-4B (new)	4B, 8-bit	Current small general vision model from Qwen; replaces Qwen2.5-VL-3B	`qwen35_1536`
Gemma 4 E4B (new)	4B effective, 8-bit	Google's 2026 small vision model, Apache 2.0	`gemma4_1536`
Chandra OCR 2 (new)	4B, 8-bit	Strong on handwriting; check its licence first	`chandra_1536`
PaddleOCR (PP-OCRv5)	Small	Non-generative control: reads characters, does not guess words	`ppocr_1536`

The three 4B models (Qwen3.5-4B, Gemma 4 E4B and Chandra OCR 2) do not fit in 7.5 GB at full precision, so load them with 8-bit weights. Record the precision and the peak memory next to every result; the paper marks these models with a dagger (†).

Do: for each model, activate its environment and run it under `vram_guard.py` exactly as in Step 2, on `data/scans` and `data/planted`. Then build the comparison table:

```
python score.py --data ../data \
  --pred ../results/glm_1536 ../results/glm_sdk_1536 ../results/paddlevl_1536
\
         ../results/deepseek_1536 ../results/qwen35_1536
../results/gemma4_1536 \
         ../results/chandra_1536 ../results/ppocr_1536 \
  --names GLM-OCR GLM-OCR-SDK PaddleOCR-VL DeepSeek-OCR-2 Qwen3.5-4B Gemma-4-
E4B \
          Chandra-OCR-2 PP-OCRv5 \
  --compare ../results/table3_ocr.csv
```

Output: one scored results folder per model and `results/table3_ocr.csv` (this becomes Table III in the paper).

Done when: every model has 28 transcripts and a memory log. A `FAIL` memory result or a model that will not load is a finding, not a failure: write the peak memory or the error message in `notes.md` and move on.

Step 5: Resolution sweep

We want to know how image size changes both memory and faithfulness. Raise the memory cap so the run measures the true peak instead of stopping; the 7.5 GB check is still recorded.

Input: the 28 photos; image sizes 1024, 1280, 1536, 2048 and 2560 px (longer side).

Do (GLM-OCR example; repeat for each model's runner):

```
for s in 1024 1280 1536 2048 2560; do
  python vram_guard.py --budget-gb 7.5 --log ../logs/glm_sweep_${s}.csv -- \
    python ocr/run_glm.py --images ../data/scans --out
../results/glm_sweep_${s} \
      --max-side $s --mem-cap-gb 11.5
  python vram_guard.py --budget-gb 7.5 --log
../logs/glm_sweep_${s}_planted.csv -- \
    python ocr/run_glm.py --images ../data/planted --out
../results/glm_sweep_${s} \
      --max-side $s --mem-cap-gb 11.5
  python score.py --data ../data --pred ../results/glm_sweep_${s} --names
GLM-${s}
done
```

Output: `results/sweep_summary.csv` with one row per model and size: `model, max_side, peak_vram_mib, median_seconds, cer_all, cer_code_table, EPR, SCR`. Take peak memory from the `[vram_guard]` line and the scores from each `scores_summary.json`.

Done when: every model has 5 rows. This table becomes the paper's memory figure.

Step 6: Choose the transcriber

Input: `table3_ocr.csv` and `sweep_summary.csv`.

Decision rule (apply in this order, at 1536 px):

1. Remove every model with EPR below 85%: it would inflate grades.
2. Remove every model whose peak memory failed the 7.5 GB check.
3. Pick the lowest CER on code and table pages (`cer_code_table`).
4. If two models are within 1 percentage point, pick the one using less memory.

Output: `results/decision.md` with the chosen model, its numbers, why each other model lost, and 2 or 3 clear examples of silent correction (page, line, what was written, what the model output). These examples become a figure in the paper.

Done when: the project lead has read and agreed with `decision.md`.

7. Real Exam Scripts

The test pages show how faithfully each model copies. Real scripts show whether the whole system grades like a teacher. We have 15 real scripts and may collect more.

Step 7: Prepare the real scripts

Input: the original answer scripts, the question paper, the official marking scheme, and permission from the course teacher or department.

Do:

1. Confirm permission first, and note the course, exam and year.
2. Give each script an ID from `s01` to `s15`. The list linking IDs to roll numbers stays with the project lead only, never in `~/research`.
3. Before photographing, cover every name, roll number, registration number and signature on every page with a paper strip.
4. If the original examiner's marks or ticks are written on the pages, do not try to remove them. Note their ink colour in the index; the project lead will remove them from the images digitally.
5. Photograph every page with the same rules as Step 1. Save as `data/real/scans/s01_p01.jpg`, `s01_p02.jpg`, and so on.
6. Photograph or scan the question paper and marking scheme into `data/real/`.
7. Fill the three spreadsheets below (save as CSV).
8. Type the exact text of about 2 pages per script (about 30 pages), choosing code and table answers first. Save as `data/real/ground_truth/s01_p02.md`. Type the student's mistakes exactly as written. Leave out anything the student crossed out.
9. While typing, record every genuine student mistake you notice in `student_errors.csv`. These natural errors let us test silent correction on real handwriting, not only on planted mistakes.

The spreadsheets (first row = column names):

`data/real/index.csv`, one row per page:

```
script,page,questions_on_page,answer_type,examiner_ink,notes
s01,1,5(a),code,red,
s01,2,5(b);6(a),code;table,red,part of 6(a) continues on page 3
```

`data/real/original_marks.csv`, the marks the original examiner gave:

```
script,question,marks,max_marks
s01,5(a),4,5
```

`data/real/rubric.csv`, the marking scheme with one row per marking point:

```
question,point,description,marks
5(a),1,Correct loop from 1 to n,1
5(a),2,Factorial variable starts at 1,1
```

`data/real/student_errors.csv`, genuine mistakes found while typing:

```
page,line,student_wrote,correct_form,type,error_code
s01_p02,7,fact = fact * i,fact = fact * i;,syntax,SYN-01
```

Use `answer_type` values `prose`, `code`, `table`, `trace`, `flowchart` or `mixed`. Use `type` values `language`, `syntax`, `logic`, `value` or `structure`. Error codes come from Appendix A of the C booklet (for example `SYN-01` missing semicolon, `IO-01` missing `&` in `scanf`, `CF-05` off-by-one loop bound).

Output: everything above in `data/real/`.

Done when: a second person has checked every photo for visible names or numbers, every page appears in `index.csv`, and the typed pages were compared with the photos word by word.

Step 8: Independent second grading

To know whether the AI grades like a teacher, we need two human marks for each answer, given independently. The difference between the two teachers is the standard the AI is compared against.

Input: the anonymised photos with the original marks removed, the question paper, `rubric.csv`, and Appendix A of the C booklet.

Do: the project lead arranges a second instructor. The instructor grades every answer using only the marking scheme, without seeing the original marks, and writes down the error codes they see.

Output:

`data/real/grades_instructor2.csv`:

```
script,question,marks,max_marks,error_codes,comment
s01,5(a),3.5,5,SYN-01;CF-07,missed semicolon and fact starts at 0
```

Done when: every answer in `original_marks.csv` also has a row in `grades_instructor2.csv`.

Watch out: if the original marks cannot be removed from the images, the second grading is not blind. Record this in `notes.md`; the paper must mention it.

8. Later Steps: Grader, Extra Tests and Paper

The project lead leads these steps; you will help run them. They are described here so you know where your Stage 1 results go.

Step	Input	Output	Feeds the paper's
9. Booklet review	The C booklet	Instructor's corrections as comments in the booklet	Claim that the knowledge base was checked by an instructor
10a. Guessing-test pages (new)	3 or 4 extra sheets of deliberately strange code (made-up names like `qzvx`, keywords like `whlie`)	Photos and ground truth, scored like Step 2	Evidence that a model reads instead of guessing
10b. Confidence flags (new)	Each transcriber's per-word confidence, saved by the updated runner	`<page>.conf.json` and a list of flagged words per page	Whether low confidence catches silent corrections and misreads
10c. Fairness check (new)	Page scores, plus a neatness rating (neat, average, untidy) for each real script	Error rates per neatness group; counts of under- and over-marking	Fairness paragraph
11. Build the grader	Chosen transcriber's transcripts, `rubric.csv`, the booklet, the error codes	`grading.py`, a booklet search index, a neutral grading prompt and the quote checker	Method section
12. Grade the real answers	Transcripts of the real scripts, both instructors' marks	`results/table4_grading.csv` comparing 5 grader setups, plus `coverage_curve.csv`	Grading results table and the safe-automation figure

Step	Input	Output	Feeds the paper's
13. Final numbers	All results files	Red placeholder values in the paper replaced	Every table and figure

What the grader will do (Step 11)

For each answer, the grader receives the transcript, the question's marking points and the most relevant booklet passages. For each marking point it returns one record:

```
{"point": "5(a)-2", "awarded": 0, "max": 1,
 "evidence": "long fact = 0;",
 "error_codes": ["CF-07"],
 "reason": "factorial starts at 0, so the product is always 0"}
```

The evidence checker then confirms that the quoted text really appears in the transcript. A record that fails twice goes to the human examiner instead of being accepted. The grader will be a small open model run by `llama-server` within the 8 GB budget, most likely Qwen3.5-9B or Qwen3.5-4B in 4-bit; Qwen3-8B is already downloaded for early tests. If time allows, we also test a small LoRA add-on trained on graded answers (upgrade 7).

The five grader setups compared in Step 12

Setup	What changes	Question it answers
G1	Marking scheme only	Baseline
G2	G1 plus booklet passages	RQ2: does the course knowledge help?
G3	G2 plus required quotes and the checker (full TRACE)	RQ3: does evidence reduce unsupported marks?
G4	G3 run on hand-typed text instead of OCR output	How many marks are lost or gained because of transcription
G5 (new)	A vision model (Gemma 4 E4B or Qwen3.5-4B) grades straight from the photo in one call	Is splitting the two stages worth it?

Each setup is compared with the average of the two instructors using quadratic weighted kappa, mean absolute error and the share of answers within 1 mark. Two more results come from G3: the safe-automation curve (accuracy against the share of answers graded

without a human) and the fairness split (how often a student is under-marked versus over-marked). Together they tell a department whether, and how far, it can rely on the system.

Watch out: research published in September 2026 found that one harsh line in a grading prompt ("never give partial credit") made most small open models grade badly. Keep every prompt neutral and tied to the marking scheme.

9. Rules to Follow

These rules keep the results trustworthy enough to publish. If a rule blocks you, ask the project lead instead of working around it.

1. **Never correct the student.** Not in the ground truth, not in the transcripts, not "just a typo". The whole project measures whether mistakes survive.
2. **Never edit or overwrite original photos.** Folders `data/scans`, `data/planted` and `data/real/scans` are read-only. Any changed image goes into a new folder.
3. **One results folder per run.** Never rerun into a folder that already has results; make a new folder or delete the old one first.
4. **Every model run goes under `vram_guard.py`** with `--budget-gb 7.5`, and its log is kept in `logs/`.
5. **Same settings for every model:** same photos, 1536 px unless it is the sweep, greedy decoding (no random sampling).
6. **Write it down.** Keep a `notes.md` in each results folder: date, exact command, memory line, scores, and anything odd. A failure you recorded is a result; a failure you forgot is lost.
7. **Commit code, not data.** After changing any script, run `git add` and `git commit -m "what changed"` in `~/research/code`. Photos and models stay out of git.
8. **Back up daily.** Copy `data/` and `results/` to an external drive or Google Drive at the end of each day. The photos cannot be recreated.
9. **Protect students.** No names or roll numbers in any file. Never upload real scripts to online AI tools or public websites; everything runs on the lab PC.
10. **Ask before changing a decision:** the 85% threshold, image size, prompts, or the ground-truth conventions. Changing them halfway makes earlier results unusable.

10. Glossary

Term	Meaning
OCR	Optical character recognition: turning an image of writing into text
VLM	Vision-language model: an AI model that reads images and writes text; most modern OCR models are VLMs
Transcript	The text file a model produces from one page photo (`p07.md`)
Ground truth	The exact text of a page, typed by a person; the answer key for scoring transcripts
Planted error	A mistake written on purpose into a copy of a test page, listed in `manifest.json`
Guessing test	A page of deliberately strange code that only a model that truly reads can copy correctly
Silent correction	A model writing the correct form instead of the student's mistake
CER / WER	Character / word error rate: the share of characters or words that differ from the ground truth
EPR	Error-preservation rate: the share of planted mistakes the model copied exactly (target 85% or more)
SCR	Silent-correction rate: the share of planted mistakes the model fixed
Confidence flag	A word the transcriber was unsure about, highlighted for the examiner
Safe-automation curve	How accurate the grader is if it grades only the answers it is most sure about, for each share of answers
Under-marking / over-marking	The system gives fewer / more marks than the teachers; under-marking harms the student
VRAM	Memory on the graphics card; our limit is 8 GB, checked at 7.5 GB

Term	Meaning
Greedy decoding	The model always picks its most likely next word, so the same input gives the same output every time
4-bit / GGUF	A compressed model format that lets a larger model fit in less memory
`llama-server`	The program that runs the grader model and answers requests from our scripts
vLLM	A fast model server; used for GLM-OCR's official SDK in Step 3
Retriever	The part of Stage 2 that finds the booklet passages relevant to a question
Marking scheme / rubric	The official list of points and marks for each question (`rubric.csv`)
Error code	A label for a common student mistake from Appendix A of the C booklet, such as `SYN-01`
QWK	Quadratic weighted kappa: agreement between two graders, from 0 (chance) to 1 (perfect)
LoRA	A cheap way to fine-tune a model by training a small add-on instead of the whole model

11. Sources

Research and model releases behind the October 2026 upgrades:

- Handwritten Code Recognition for Pen-and-Paper CS Education (Stanford): "logical fix hallucination" in 11 handwritten Python programs
- Automated Grading of Handwritten Mathematics Using Vision-Capable LLMs (Levine et al., University of Illinois, May 2026): transcription errors caused 87% of disagreements
- When Low CER is Not Enough (Gardella et al., September 2026): OCR models silently normalise spelling in historical documents
- Visual Merit or Linguistic Crutch? A Close Look at DeepSeek-OCR (January 2026): reliance on language guessing; code untested
- Reading or Guessing? (Karamolegkou et al., September 2026): perturbation tests for OCR faithfulness

- Where LLM Graders Succeed and Break (Habibullah et al., September 2026): prompt harshness and LoRA results for small open graders
- Towards Fully Automated Exam Grading (Grabowski, June 2026): fairness-aware grading of handwritten answers
- CHiL(L)Grader (Raikote et al., March 2026): calibrated human-in-the-loop grading
- Semantic Grading of Written Answers in Bangla (Farzana et al., University of Dhaka, June 2026): local Bangla–English grader
- PaddleOCR-VL-1.5 model card
- Best Open-Source OCR Models in 2026 (Roboflow)
- Chandra 2 OCR model overview
- Qwen3.5 open-weights family (DeepLearning.AI)
- Gemma 4 release notes (Simon Willison)
- Bangladesh draft National AI Policy (The Financial Express)